codegraph_scope_guard.py — guide

Revision: 1 **Last modified:** 2026-09-26T13:03:00Z

Source: `constitution/scripts/codegraph/codegraph_scope_guard.py` (433 lines). Tests: `constitution/scripts/codegraph/tests/run_scope_tests.sh`, which runs `t_scope_cases.py` (cases T01-T17) and `t_scope_mutations.py` (mutants M0-M15, 18 results) against a generated fixture repository. Citations below are `codegraph_scope_guard.py:<line>` unless another file is named.

Overview

Proves that the file set a CodeGraph index would contain for a project is the mandated scope (header, lines 3-12): third-party submodules out, own-organisation submodules in, first-party source indexable, secret and bulk classes out. It does not model the engine: it asks the installed runner's own file discovery (`scanDirectory`) through `scope_enumerate.js` and judges the answer (lines 5-6, 97-108). It never opens a `.codegraph` database or lock and never runs `codegraph init/index/sync` (lines 32-34).

The checks (violation ids, header lines 39-41):

Check id	Fires when	Source
`third_party_files`	any enumerated file lies under a third-party submodule root	lines 128-139, 233-238
`forbidden_class`	any enumerated path matches a `forbidden_classes` regex from the scope file	lines 246-250
`negation_target_empty`	a `reinclude_negations` pattern matches no enumerated file	lines 252-260
`include_target_empty`	an `include_patterns` pattern matches no enumerated file	lines 262-269
`own_org_empty`	an own-org submodule is checked out and has tracked source but contributes zero enumerated files	lines 271-278
`stale_render`	`codegraph.json` / the `.gitignore` block on disk differ from a fresh render of the scope	lines 225-231
`count_out_of_tolerance`	the enumerated count is outside `accepted_count` +/- `tolerance_pct`, or `accepted_count` is unset (0)	lines 280-287
`control_needle`	the instrument could not prove it can see (see below)	lines 199-223
`runner_contract`	the runner no longer behaves as this tooling assumes	lines 379-388
`render_nondeterministic`	in dry-run mode, two renders of the same scope differ	lines 395-398

Every "zero" is control-needled (§11.4.273): a blind instrument yields exit 4, never a clean pass (header lines 11-12).

Prerequisites

- `python3` with PyYAML (loaded by `scope_render.py`, `scope_render.py:73-77`), `node` >= 18 (`SCOPE_NODE` overrides the binary, line 99), `git`.
- Sibling files in the same directory: `scope_render.py` (imported, line 59) and `scope_enumerate.js` (line 61).
- A scope file (`scope.example.yaml` documents the schema) and a project root that is a directory.
- A runner `lib/dist` (see "Runner resolution").

Usage

```
codegraph_scope_guard.py --scope <scope.yaml> --root <project root>
    [--dist <runner lib/dist>] [--work <scratch dir>]
    [--print-count] [--contract-only]
    [--dry-run-dir <dir>] [--explain] [--report-out <file>]
codegraph_scope_guard.py --resolve-dist [--dist D] [--scope S]
```

Flag	Meaning	Source
`--scope FILE`	Scope file. Required except with `--resolve-dist`	lines 322, 361-362
`--root DIR`	Project root. Must be a directory	lines 323, 363-365
`--dist DIR`	Runner `lib/dist` (see below)	lines 324, 355
`--work DIR`	Scratch directory. Default: a fresh `scope_guard_XXXXXX` directory under `$TMPDIR`. Refused (exit 3) when inside `--root`	lines 325, 366-370
`--print-count`	Print the enumerated count alone on stdout; the count-acceptance check is skipped	lines 326, 281, 419-420
`--con-tract-only`	Run only the runner-contract probe, then exit	lines 327, 389-390
`--re-solve-dist`	Print the resolved runner dist path and exit 0	lines 328, 356-358
`--dry-run-dir DIR`	Render the scope into DIR and enumerate with the rendered files overlaid; nothing is written to the root. Refused (exit 3) when inside `--root`	lines 329, 185-187, 291-303, 367-369
`--ex-plain`	With `--dry-run-dir`: also list tracked source still dropped by the engine's built-in skips, into `<dir>/explain_dropped.tsv`. Silently ignored without `--dry-run-dir`	lines 330, 401-412
`--re-port-out FILE`	Also write the report JSON to FILE (atomic write)	lines 331, 344-345
`-h`, `--help`	argparse help, exit 0	lines 333-335

Runner resolution

First usable of: `--dist`, the scope's `runner.dist_dir`, `$CODEGRAPH_RUNNER_DIST`, then the `codegraph` executable on `PATH` (looking for `node_modules/@colbymchenry/codegraph-*/lib/dist`, `lib/dist` and `../lib/dist` relative to the executable's real directory). A usable dist has `extraction/index.js` and `project-config.js`. An explicit choice that is unusable is an error (exit 3): it never falls through to the next source (lines 71-94).

Examples

#	Command	Status
1	`codegraph_scope_guard.py --help`	VERIFIED (ran; saw the usage text listing all 10 options, exit 0)
2	`codegraph_scope_guard.py --resolve-dist`	VERIFIED (ran; saw `.../node_modules/@colbymchenry/codegraph/node_modules/@colbymchenry/codegraph-linux-x64/lib/dist`, exit 0)
3	`code-graph_scope_guard.py` (no arguments)	VERIFIED (ran; saw `scope_guard: FAIL-CLOSED: --scope and --root are re-quired` and a `SCOPE_GUARD_REPORT` line with `"exit_code": 3`, exit 3)
4	`codegraph_scope_guard.py --bogus`	VERIFIED (ran; argparse `unrecognized arguments: --bogus`, exit 3)
5	Guard on the generated, un-rendered test fixture (`--scope <fixture>/code-graph.scope.yaml --root <fixture> --work <scratch>`)	VERIFIED (ran; saw `scope_guard: FAIL(rc=1) count=13 viola-tions=['count_out_of_tolerance', 'for-bidden_class', 'include_target_empty', 'negation_target_empty', 'stale_render', 'third_party_files']`, exit 1)
6	Same fixture after `scope_render.py --write`, with `--print-count`	VERIFIED (ran; stdout `8`, stderr `scope_guard: PASS count=8 viola-tions=none`, exit 0)
7	Same fixture with `accep-ted_count: 8` set and re-rendered, plain run with `--report-out`	VERIFIED (ran; saw `PASS count=8`, exit 0; report keys `contract controls count enu-meration_file exit_code include_targets mode negation_targets own_org root run-ner scan_mode scope_summary third_party tool verdict violations`; `verdict PASS`, `mode check`, `scan_mode git`)

#	Command	Status
8	`--dry-run-dir <dir> --explain --print-count` on a fresh un-rendered fixture	VERIFIED (ran; saw `PASS count=8`, exit 0; `<dir>` held `codegraph.json`, `explain_dropped.tsv`, `gitignore.block`, `gitignore.overlay`; the fixture root's file listing/size/mtime hash and its `git status` hash were identical before and after)
9	`--work <dir inside root>` and `--dry-run-dir <dir inside root>`	VERIFIED (ran; saw `FAIL-CLOSED: --work ... is inside the project root`, exit 3; nothing created in the root)
10	unparsable scope file / `--root` that is not a directory / `--dist /nonexistent/dist`	VERIFIED (ran; `unparsable scope file ...`, `root is not a directory`, `--dist='/nonexistent/dist' is not a runner lib/dist`; exit 3 each)
11	`--contract-only`	VERIFIED (ran; exit 0; report `mode: contract_only`, `contract.ok: true`, runner version `1.6.0`, checks `C0_config_filename` ... `C5_exclude_wins_over_negation`)
12	Guard against this repository's real project root	NOT RUN (deliberately: the brief forbids pointing anything at the live 580K-file tree or its index; the guard would enumerate it)

Inputs

- The scope file (parsed and validated by `scope_render.load_scope`), rendered files on disk (`codegraph.json`, `.gitignore`), the project's git tree and `.gitmodules` files, and the runner dist.
- Scope sections the renderer does not validate and the guard requires (lines 148-168):
  - `forbidden_classes`: non-empty list; each entry needs string `name`, `regex`, `positive`, `negative`; the regex must compile;

    - `enumeration_controls.present` / `.fabricated`: lists of paths (optional);
    - `accepted_count` (integer >= 0, default 0) and `tolerance_pct` (number >= 0, default 0). Any violation of these is a fail-closed exit 3 (lines 151-167).
- Environment: `CODEGRAPH_RUNNER_DIST`, `SCOPE_NODE`, `TMPDIR` (default work directory, line 366).

Outputs

- stderr: one line `SCOPE_GUARD_REPORT <json>` (keys sorted, no timestamps or durations, lines 337-346), a human line `scope_guard: PASS|FAIL(rc=N) count=<n> violations=<list>` (lines 427-428), and on a fail-closed exit `scope_guard: FAIL-CLOSED: <reason>` (line 414).
- stdout: only the count with `--print-count` (line 420) or the dist path with `--resolve-dist` (line 357). The count is printed even when the run fails on violations (example: `--print-count` on the un-rendered fixture printed `13` and exited 1), but not on a fail-closed (3) or contract (2) exit.
- Report fields (lines 337-412): `tool`, `mode` (`check` / `dry_run` / `contract_only`), `root`, `verdict`, `exit_code`, `violations`, `controls`, `contract`, `scope_summary`, `count`, `enumeration_file`, `runner` (`version`, `config_filename`), `scan_mode` (`git` or `walk_fallback:<reason>`), `third_party` (`roots`, `files_enumerated`, `exclusion_proven_on_roots`, `vacuous_roots`), `own_org`, `negation_targets`, `include_targets`; in dry-run also `stale_render` (`not_applicable_dry_run`), `reinclude_untracked_ignored`, `explain`; on exit 3, `error`.
- Files: `<work>/contract/`, `<work>/roots.json`, `<work>/enumerated.txt` (lines 176-182, 376-378), and `--report-out`.

Exit codes

Code	Meaning	Source
0	Scope proven (no violations); also `--help`, `--resolve-dist`, `--contract-only` success	lines 335, 358, 390, 426
1	One or more violations, none of them a `control_needle`	lines 423-424
2	Runner contract drift: the engine no longer behaves as assumed	lines 384-388
3	Fail-closed: unparsable or invalid scope, unloadable runner, bad root, unsafe path (`--work` / `--dry-run-dir` inside the root), enumeration or explain failure, usage error (also argparse errors)	lines 334-335, 413-416
4	Control needle blind: any `control_needle` violation wins over every other violation	lines 421-422

Edge cases

- **Order of work.** The runner contract is probed FIRST (lines 374-391); if it drifts the guard exits 2 and measures nothing else.
- **Control needles** (lines 199-223): each forbidden class's `positive` needle must match its regex and its `negative` needle must not; the enumeration must be non-empty; every `enumeration_controls.present` path must be enumerated; no `enumeration_controls.fabricated` path may be. Any failure is a `control_needle` violation, hence exit 4 (tests T08, T09).
- **Third-party proof can be vacuous.** `third_party.vacuous_roots` lists third-party roots that are not checked out or have no tracked source, for which "zero enumerated" proves nothing; this is reported, not a violation (lines 239-244).

- **Count acceptance.** With `accepted_count` 0 every non-`--print-count` run fails `count_out_of_tolerance` ("measure with --print-count, review, then set it"); otherwise the check is `abs(count - accepted) * 100 > tolerance_pct * accepted` (lines 281-287).
- **Own-org roots not checked out** are listed under `own_org.not_checked_out` and are not a violation (line 274).
- **Scan mode is measured, not assumed.** If the runner's `git ls-files` listing throws (for example a buffer overflow on a huge tree) the runner silently falls back to a filesystem walk; the report says `walk_fallback:<code>` because the file-set semantics then differ (`scope_enumerate.js:136-149` ; test T17).
- **The guard deletes inside `--work` .** It removes `<work>/enumerated.txt` and recursively removes `<work>/contract` before recreating them (lines 177-178, 376-378). Verified: a pre-existing `<work>/contract/precious.txt` was deleted and a pre-existing `enumerated.txt` was overwritten. Point `--work` at a dedicated scratch directory.
- **Default work directory is not cleaned up.** Without `--work` a `scope_guard_XXXXXX` directory (holding `contract/` , `enumerated.txt` , `roots.json`) is created under `$TMPDIR` and left behind (line 366; observed).
- **Dry run.** Renders twice to prove determinism (lines 395-398), writes overlay files only under `--dry-run-dir` , and lets the runner read them in place of `<root>/codegraph.json` and `<root>/.gitignore` (`scope_enumerate.js:57-85`). `reinclude_untracked_ignored` counts files git itself ignores under literal, anchored negation directories; a non-literal pattern is reported as `unmeasured_non_literal_pattern` (lines 306-317).
- **`--resolve-dist` with a bad scope** still prints the dist path: an unparsable scope is only raised after the `--resolve-dist` branch (lines 349-360).

Internal behaviour

1. Parse arguments; load the scope for the runner settings; resolve the dist (lines 332-358).
2. Validate `--scope` / `--root` , choose and check `--work` / `--dry-run-dir` (lines 359-372).

3. Runner contract probe through `node scope_enumerate.js contract` (lines 374-391).
4. Render the scope (`scope_render.render`); in dry-run mode render twice and write the overlay files (lines 392-399).
5. `evaluate` (lines 172-288): enumerate through `scope_enumerate.js enumerate` with a roots file (own plus third-party roots), then run the control needles, stale-render, third-party, forbidden-class, negation, include, own-org and count checks.
6. Dry-run extras: `reinclude_untracked_ignored` and optional `explain` (lines 401-412).
7. Decide the exit code, print the report (lines 418-429).

Related

File	Relation
scope_render	Renders `codegraph.json` and the `.gitignore` block; the guard imports it
scope_enumerate	The node bridge into the runner's own discovery
`scope.example.yaml`	Scope schema example
`scope_baseline.txt`	Must-exclude classes data file for the `codegraph_safe.sh` scope-proof stage (its header, lines 1-3); this guard does not read it
codegraph_safe, codegraph_guard, disk_tripwire	Sibling tools that refer to this guard as the scope proof before "ready"
`tests/run_scope_tests.sh`, `t_scope_cases.py`, `t_scope_fixture.py`, `t_scope_mutations.py`	Test suite and fixture builder
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26T13:03:00Z — everything ran against generated fixtures in a scratch area; the real repository and its `.codegraph` were never opened.

- `TMPDIR=<scratch> bash tests/run_scope_tests.sh` (1 iteration): T01-T17 `PASS` (`SUMMARY cases=17 fail=0`), M0-M15 `PASS` (`SUMMARY mutations=18 fail=0`), `DETERMINISM iterations=1 ... verdict=PASS` . (One iteration proves no cross-iteration determinism; `--iterations N` exists for that, line 26 of the runner, and was not run.)

- The direct invocations in the examples table, on scratch fixtures built by `tests/t_scope_fixture.py`.